sw-smoke

An AgentMPI run analysis

Abstract

This is the first software-experiment run in the archive, recorded at 07:02:01 against commit `b0b652d`, before any real-agent run existed. Four ranks under the deterministic surrogate executor take two `minidb` modules each through one round of the pipeline in 0.119 s: broadcast the specification, publish eight interfaces into the RMA window, fence, read the fourteen dependency slots the module graph requires, implement, barrier, gather to rank 0, run the acceptance oracle, broadcast the report, scatter the blame, agree. All six checkable collectives sent exactly the number of messages their closed forms predict, no contract was violated, and no rank failed. That is the entire result: the pipeline runs end to end at $p = 4$. It is not a measurement. The executor is `function`, so the sixteen agent calls are dictionary lookups, the tree they produce is 24 lines of stubs scoring 0/60 by construction, and the $0.094 in the headline table prices prompts no model read. Its one incidental contribution is coverage: at $p = 4$ the harness takes the `linear` gather and scatter path rather than the binomial one every larger software run uses, and this run and its twin `sw-vague-smoke` are the only places that path is exercised.

1 What this run is

property	value
run	`sw-smoke`
experiment	`software`
campaign label	`sw-smoke`
declared world size	4
ranks appearing in the log	4
executors	`function` × 4
events	128
wall time	0.12 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 4 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	61.7%	rank-seconds blocked in collectives, of those available
coordination in flight	83.2%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	16	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	20	17 eager, 3 rendezvous
tokens sent	5,642	189 deferred under rendezvous
tokens in / out	28,965 / 496	prompt and completion
cost	$0.0943	0.1902 $/kTok out

3 What the analysis notices

- [note] Collectives account for 61.7% of the run's rank-seconds, so more of the pool's time went to coordination than to work.
- [ok] All 6 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.

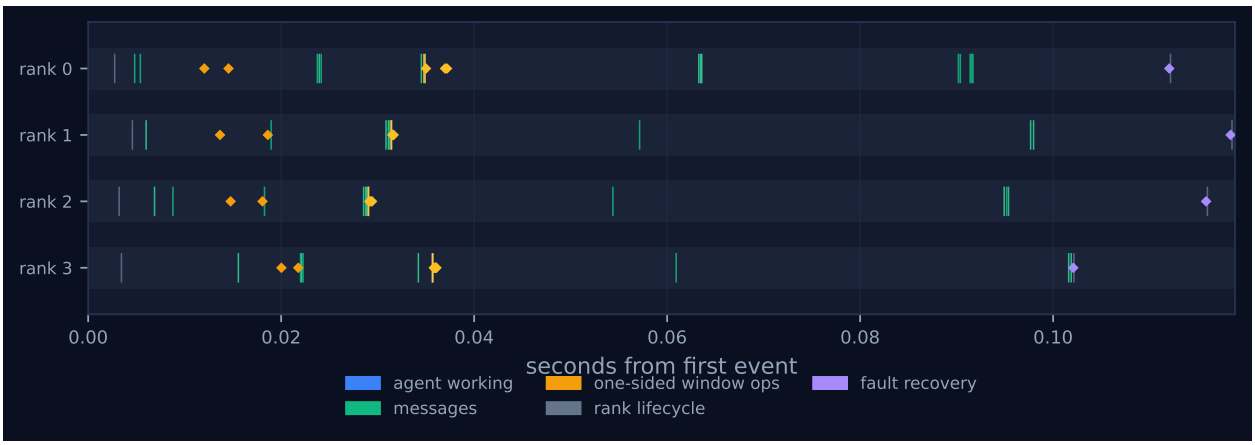

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

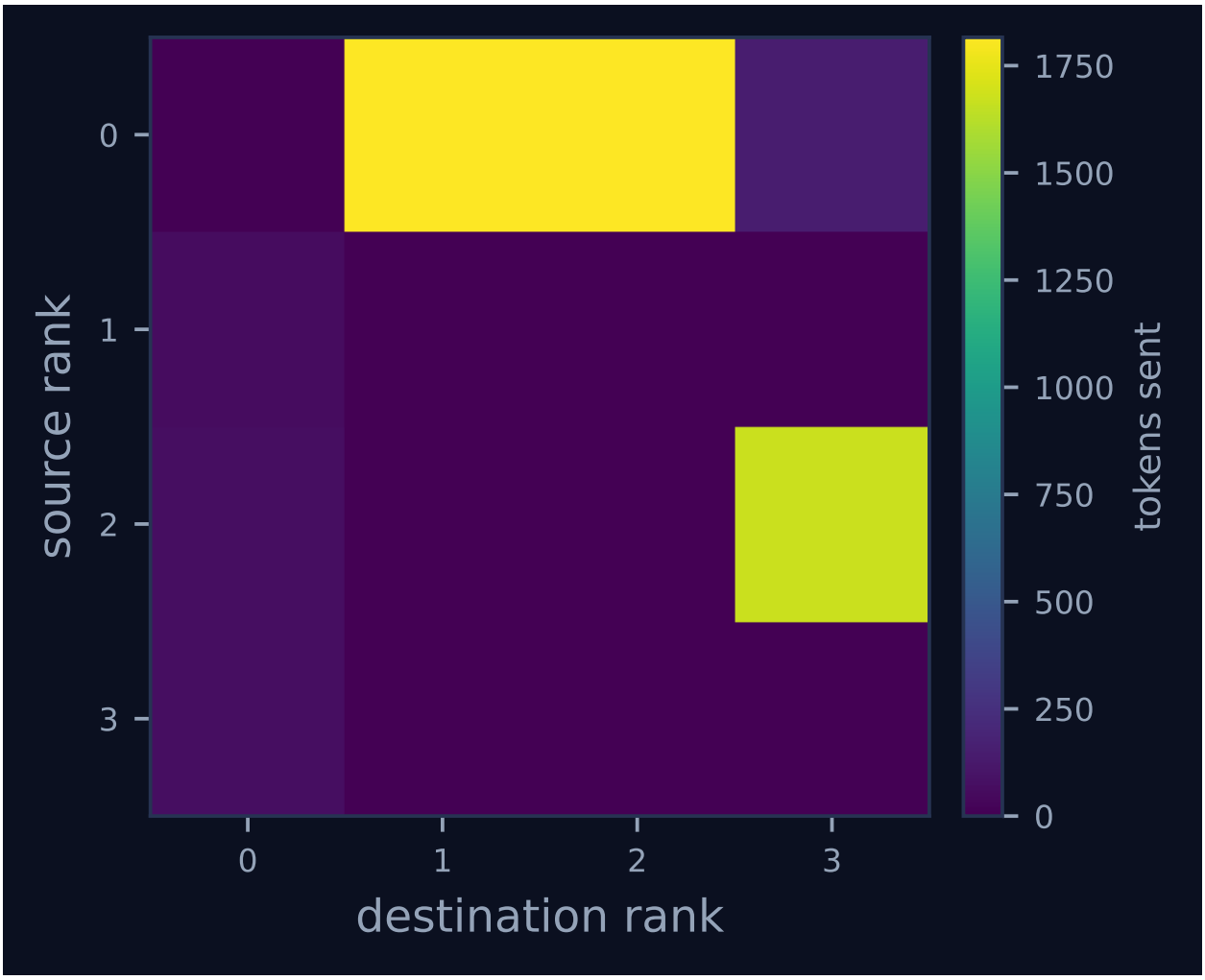

Figure 2: Communication matrix: tokens sent from each rank to each rank. The algorithm's shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

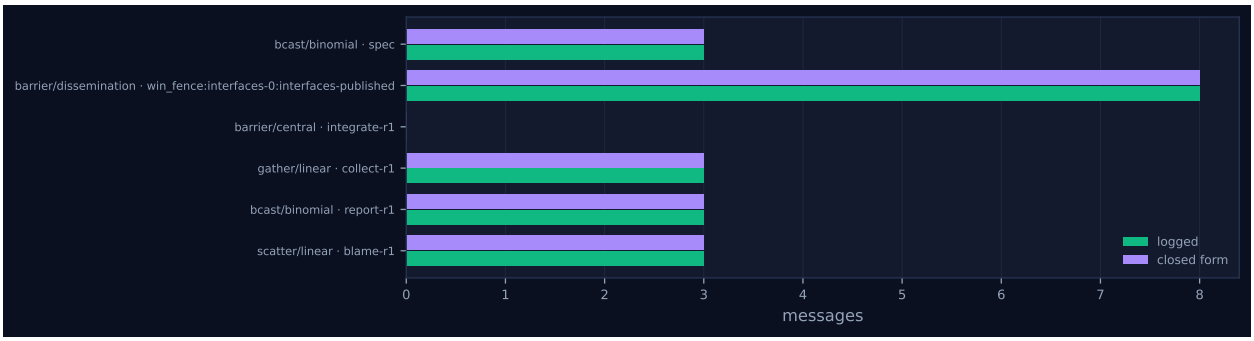

Figure 3: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	4	9	5	3,779	0.0	finalized
1	0.00	0.0	0	4	3	5	65	0.0	finalized
2	0.00	0.0	0	4	5	5	1,733	0.0	finalized
3	0.00	0.0	0	4	3	5	65	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)
bcast	binomial	spec	4	4	2	2	3	3	4,578	0.01
barrier	dissemination	win_fence:interfaces-0:interfaces-published	4	4	0	2	8	8	8	0.02
barrier	central	integrate-r1	4	4	0	2	0	0	0	0.02
gather	linear	collect-r1	4	4	1	3	3	3	189	0.02
bcast	binomial	report-r1	4	4	2	2	3	3	426	0.04
scatter	linear	blame-r1	4	4	1	3	3	3	441	0.00

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has no work bars, and the absence is the first thing to explain. Under `-executor function` every `agent.call` carries `latency_s: 0.0`, so all sixteen calls are instants and the four lanes hold nothing but message ticks, amber window diamonds and four violet `ft.agree` marks at the right-hand edge, across a span of 0.119 s. Everything the headline table says about occupancy follows from this and none of it is about scheduling: achieved parallelism 0.00×, parallel efficiency 0.0 %, load imbalance 0.00× and idle fraction 100.0 % all report that no rank was ever measurably busy. The [note] that collectives take 61.7 % of rank-seconds is arithmetically true and interpretively empty — with zero work seconds, coordination is the only quantity in the ratio.

The phase sequence is what remains legible, and it is the harness's. Four `rank.init`s and the `spec` broadcast open the run; eight `iface` calls, each immediately followed by its `win.put`, carry it to $t = 0.022$ s; the `interfaces-published` fence releases each lane in turn, and each then issues a `win.sync` and its own dependency reads — five for rank 3, which owns `functions` and `api`, three for rank 0, which owns `errors` and `parser`; eight `impl:*:r1` calls follow, then the `integrate-r1` barrier and the gather at $t \approx 0.06$ s, the report broadcast and blame scatter at $t \approx 0.09$–0.12 s, and finally agreement and finalisation. Watching that complete without a stall, a timeout or a contract rejection is what a smoke test is for. Figure 2 shows the expected two-level binomial tree over a

nearly empty plane: rank 0 sends 1,816 tokens to each of ranks 1 and 2 in four messages apiece, rank 2 forwards 1,669 to rank 3 as the broadcast's second level, and the fan-in is 63 to 64 tokens per rank because the gathered modules are one-line stubs.

7 Interpretation

The configuration is `-ranks 4 -rounds 1 -executor function`, with shared interfaces, locks and review nominally enabled. Module assignment is $i \bmod p$, so the eight modules divide evenly two per rank: rank 0 owns `errors` and `parser`, rank 1 `tokens` and `planner`, rank 2 `nodes` and `engine`, rank 3 `functions` and `api`. Within that scope the run is clean — six collective invocations, all complete, all matching their predicted message counts (3 for each binomial broadcast, 8 for the dissemination fence, 0 for the central barrier, 3 each for the gather and scatter), twenty messages in total, and zero contract violations across sixteen contract-checked calls spanning both the `Interface` and `Implementation` contracts.

`-rounds 1` is the easily missed part of that configuration and it bounds the coverage sharply. Both the cross-review phase and the interface renegotiation are guarded by `round_no < cfg.rounds`, false in a single-round run, so neither executes: the trace contains no `topo.graph_create`, no `coll.neighbor_allgather` and no `win.lock` or `win.unlock` at all, and every window slot ends at version 1. The sixteen window operations are the publish-and-read half of the RMA protocol only. So although `review` and `locks` are both true in the config, the two coordination mechanisms most specific to this experiment are untested here; `sw-rev-smoke`, at $p = 6$ with two rounds, is the run that covers them.

The one thing this run exercises that no larger software run does is the small-p algorithm choice. `algorithms.gather` selects `"linear" if p <= 4 else "binomial"` and `scatter` follows the same rule, so ranks 1, 2 and 3 send straight to the root instead of folding through a tree; `sw-rev-smoke` at $p = 6$ and every real-agent run at $p = 8$ take the binomial branch. In the collectives table the `rounds` column reads 1 for both linear invocations against a prediction of 3, while the message counts agree at 3. That is not a defect: a linear gather issues all $p - 1$ transfers without waiting and counts one round, whereas the closed form counts $p - 1$ because they serialise at the root. The model check compares messages and reads 6 of 6.

Against `sw-vague-smoke`, the same configuration under the other specification regime, the two traces are the same trace: 128 events each, identical `kind_counts` and `role_counts`, the same six collectives with the same algorithms and 20 messages, the same 16 agent calls, and identical completion token totals of 496. That identity is the expected outcome rather than a curiosity — `synthetic_agent` keys its reply on the call label and ignores the prompt, so no branch can depend on what the specification says — and confirming it is a modest but real check: had the vague substitution changed the number of window operations or collectives, something in the harness would be reading the specification where it should not. Every difference is token volume and it reconciles exactly. `apply_vague_layout` makes the specification 54 characters and 15 tokens longer (1,526 against 1,541); it is embedded in all sixteen prompts, giving the 228-token difference in `tokens_in` (28,965 against 29,193) at 14.25 tokens per prompt, and broadcast to three ranks, giving the 45-token difference in the `spec` collective.

One difference between the pair is not about the specification at all. This run's oracle reports `n_total:  59` and its import traceback names line 174 of `acceptance.py`; `sw-vague-smoke`'s reports 60 and line 181. The suite gained a case in the 65 minutes between them, so the two smokes

differ in oracle version as well as in regime and are not a controlled pair. The archived offline re-scoring, which uses a single suite at `0eaa6b695b83`, reports 60 cases for both.

8 Threats to this reading

The governing threat is that none of this is a measurement. `executors: function` × `4`: the sixteen agent calls are `synthetic_agent` returning canned dictionaries keyed on the call label, the 0.119 s wall time is SQLite writes and event logging, and the $0.0943 and 28,965 prompt tokens price prompts that were genuinely assembled and counted but never sent. No statement about agent behaviour, model latency, code quality or the value of shared interfaces can be drawn from this run, and the 0.19019 $/kTok-out figure in `metrics.json` is a real prompt bill divided by 496 stub completion tokens.

The acceptance result is likewise a construction, not a failure. The oracle reports 0 of 59 with `ImportError: cannot import name 'query' from 'minidb.api'` because the surrogate writes `def stub_api(): return None`. The verdict is correct — the tree really does not import — and it is what gives the blame scatter and the agreement non-trivial payloads; reading the subsequent `agree(false)` as a negative result would be wrong.

Two smaller cautions. The four `ft.agree` events are counted under the recovery role, so `role_counts` shows 4 recovery events against 0 trouble events and the dashboard reads "fault recovery 4" beside FAILURES 0; nothing was recovered from, the agreement recording `voters: [0, 1, 2, 3]` and `excluded: []`. And $p = 4$ divides the module count evenly, so every rank owns exactly two modules and the run never encounters the uneven ownership $p = 6$ produces — it is blind to load imbalance both by that and by the surrogate's zero-cost calls.